qPCRtools: a cross-platform C++ desktop application for qPCR data analysis with validated equivalence to its R implementation

Xiang Li[1,2,*] Huichuan Huang[1,2,*]

[1] State Key Laboratory for Conservation and Utilization of Bio-Resources in Yunnan, Yunnan Agricultural University, Kunming, China

[2] Key Laboratory for Agro-Biodiversity and Pest Control of Ministry of Education, Yunnan Agricultural University, Kunming, China

[*] Corresponding authors: lixiang117423@gmail.com (X.L.), absklhhc@gmail.com (H.H.)

Present address (X.L.): Yazhouwan National Laboratory, Sanya, Hainan, China

Abstract

Summary: Quantitative real-time PCR (qPCR) is a standard technique for measuring gene expression, yet turning instrument-exported C_q values into publication-ready results requires several non-trivial steps: estimating amplification efficiency, choosing a quantification method, testing groups statistically, and preparing figures. Established tools for this chain are predominantly programming-based—including our own R package *qPCRtools*—and remain out of reach for students and wet-lab researchers without programming experience, whereas instrument software, free stand-alone programs and web applications each cover only part of the workflow or tie the analysis to one platform. We present *qPCRtools*, a cross-platform desktop application written in C++17 with Qt6 that exposes the analytical core of the R package through a point-and-click interface and ships as a single executable requiring no R, Python or other runtime. It implements standard-curve construction and efficiency estimation, the ΔC_t, $\Delta\Delta C_t$ and efficiency-based (RqPCR) quantification methods, group comparison by Welch t-test, Wilcoxon rank-sum test or one-way ANOVA with Tukey HSD letters, quality-control warnings, in-interface figures, and a plain-text record of the inputs and parameters of each analysis. On identical inputs the C++ implementation agrees with the R package to floating-point precision for every shared method; preparing the port also revealed and corrected several defects in result aggregation, each covered by a regression test.

Availability and Implementation: *qPCRtools* is written in C++17 with Qt6 and built with CMake; the core algorithm library requires only Qt Core and optionally links Eigen3, GSL and OpenXLSX. Version 1.2.0 is distributed as a single self-contained executable for Windows and macOS under the MIT license at `https://github.com/lixiang117423/qPCRtools_cpp`, with pre-built binaries in the repository releases; no registration is required, and the source compiles on Linux. The software and its source will remain available for at least two years.

Contact: `lixiang117423@gmail.com`

Supplementary Information: Supplementary Figures S1–S2 give the full analysis output and the equivalence comparison, and Supplementary Table S1 summarises the equivalence across all shared methods.

1 Introduction

Quantitative real-time PCR (qPCR) is among the most widely used techniques in molecular biology for quantifying nucleic acids and measuring gene expression [5]. A sound analysis involves

several interdependent steps: estimating primer amplification efficiency, choosing a quantification strategy, testing groups statistically, and preparing figures [4]. When target and reference efficiencies are comparable, the $2^{-\Delta\Delta C_T}$ method [8] is convenient; when they differ, an efficiency-corrected or standard-curve-based approach is required [11]. These choices, combined with the need for rigorous statistics, make qPCR analysis demanding for many laboratory researchers.

A range of tools cover different parts of this workflow (Table 1). Scripting-based implementations—our own *qPCRtools* [7] and *pcr* [1]—implement the calculation chain transparently but require programming skill and a configured environment. Software distributed with real-time instruments (for example QuantStudio Design & Analysis and CFX Maestro) offers a graphical workflow but is tied to that instrument. Free stand-alone programs cover single steps: REST [12] computes efficiency-based relative expression for two-group comparisons with a randomisation test, LinRegPCR [13, 15] estimates baseline and efficiency from raw fluorescence, and qBase [6] implements relative quantification with the geNorm stability measure [17]. Free web applications—SATQPCR [14], DAG Expression [3] and Auto-qPCR [9]—integrate several steps but require uploading the data to a remote server or deploying the web stack locally. The gap is therefore not a missing algorithm but a missing delivery mode: *validated numerical equivalence* to a published implementation combined with a *native, cross-platform, zero-dependency* graphical workflow.

Table 1: Delivery mode and functionality of representative qPCR analysis tools. • provided, ○ not provided, ○/• partial or version-dependent (statistics were added in the later qBase+ release; vendor software differs between instruments and versions). Compiled from the cited papers, the vendors' documentation and the survey of Pabinger *et al.* [10]. [a] *qPCRtools* [7], *pcr* [1]. [b] QuantStudio Design & Analysis, CFX Maestro. [c] SATQPCR [14], DAG Expression [3], Auto-qPCR [9].

Tool	Free	No coding	Offline	Efficiency	Statistics	Ref. genes
R packages[a]	•	○	•	•	•	•
This application	•	•	•	•	•	•
REST	•	•	•	•	•	○
LinRegPCR	•	•	•	•	○	○
qBase	•	•	•	•	○/•	•
Instrument software[b]	•	•	•	•	○/•	○
Web apps[c]	•	•	○	•	•	•

Statement of need

The programming requirement is not a marginal obstacle. In our own teaching and collaborative work, the users who most need a defensible qPCR analysis are frequently those least able to install an R environment and write the few lines of code the analysis requires: newly enrolled students, visiting researchers, and wet-lab staff whose projects run for a single season. They fall back either on manual spreadsheet calculations—difficult to audit and easy to get wrong—or on whichever instrument software happens to be installed. Our R package [7] provides the algorithms, and its adoption since 2022 shows that the analytical choices it implements are accepted in published work, but that adoption is bounded by the R environment itself.

We therefore reimplemented the analytical core as a native desktop application, keeping the validated behaviour of the R implementation while removing every prerequisite except a double-click. *qPCRtools* makes three contributions: (i) a C++/Qt architecture in which a tested core algorithm library is decoupled from a web-based user interface; (ii) a validation demonstrating numerical equivalence to the R package across all shared methods, shipped as a reproducible validation package; and (iii) statistical and functional extensions beyond the R version (non-parametric tests, effect sizes, outlier removal, quality-control warnings, automatic reference-gene

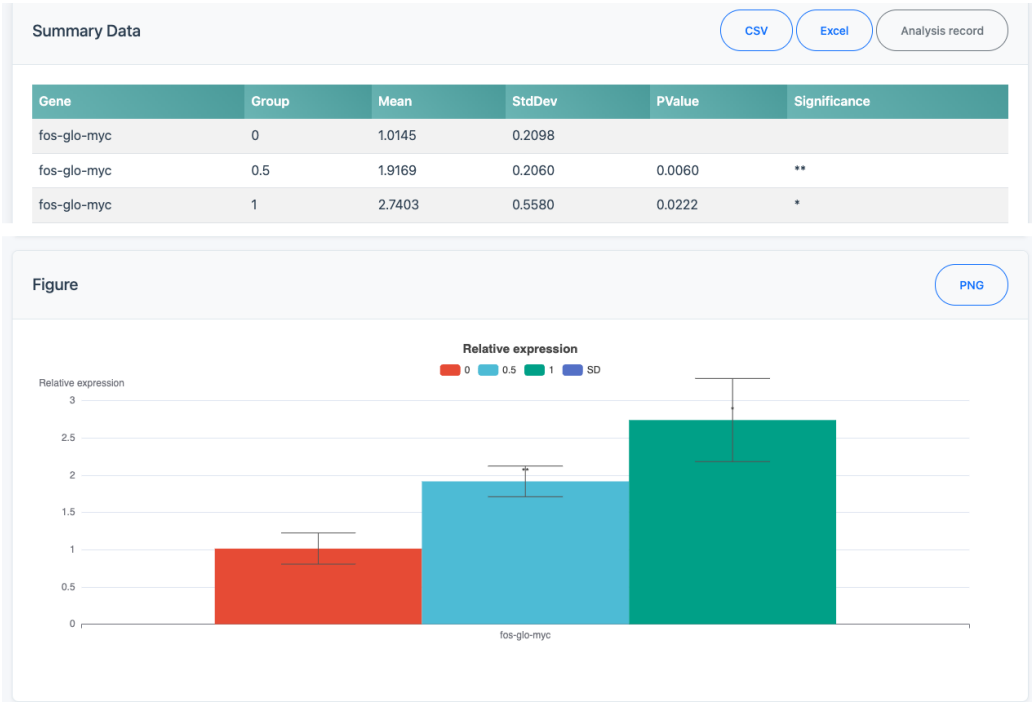


Figure 1: The *qPCRtools* desktop interface (screen capture of version 1.2.0). A single cross-platform window hosts the web-based UI, giving access to the four analysis areas (standard curve, $\Delta C_t/\Delta\Delta C_t$, statistical tests and data-entry templates). Both parts are the results page of the bundled example—the target gene *fos-glo-myc* normalised to *Beta Actin*, three dose groups with three biological replicates each—analysed by the $\Delta\Delta C_t$ method with Welch tests against the control, the two parts being cropped from one window scrolled to two positions (upper part: summary table; lower part: the in-interface figure). The summary table reports the mean relative expression, its standard deviation, the p-value and the significance symbol for each group; the figure redraws the same means with standard-deviation error bars and the significance symbols. The interface is bilingual (Chinese/English); the English language is shown here.

selection). Preparing the port also revealed defects in result aggregation shared with the R package—the input of the reference-gene stability measure, replicate handling in the efficiency method, and the $\Delta\Delta C_t$ calibrator—as well as one specific to the C++ code (multiple-comparison p-values); all were corrected and are covered by regression tests.

2 Implementation

2.1 Architecture

qPCRtools is written in C++17 and built on Qt6 [16]. The user interface is rendered with web technologies—Bootstrap 5.3 for layout and Apache ECharts [2] for figures—inside a `QWebEngineView`, and exchanges data with the native backend through Qt WebChannel. All numerical computation therefore stays in compiled code while the presentation layer remains plain HTML, CSS and JavaScript. The analytical core is a standalone static library, `qpcr_core`, that depends only on Qt Core (plus optional Eigen and GSL) and is independent of the GUI, so it is exercised directly by the unit tests and by the command-line equivalence driver. Figure 1 shows the interface at the end of a complete analysis.

2.2 Quantification algorithms

The core reimplements the methods of the R package. Amplification efficiency is obtained by ordinary least squares of C_q on $\log_d$(concentration), i.e. the number of d-fold dilution steps (d = dilution factor, 4 in the bundled example), yielding slope s, intercept, R^2 and a regression p-value; efficiency is

$$E = d^{-1/s} - 1, \tag{1}$$

with s expressed per $\log_d$ unit (equivalently, per dilution step). Relative expression is computed by the ΔC_t and $\Delta\Delta C_t$ methods [8],

$$\mathrm{RE} = 2^{-\left[(C_{q,\mathrm{target}} - C_{q,\mathrm{ref}})_{\mathrm{treated}} - (C_{q,\mathrm{target}} - C_{q,\mathrm{ref}})_{\mathrm{control}}\right]}, \tag{2}$$

and by an efficiency-based (RqPCR) method [14] in which per-sample quantities are $E^{\,C_{q,\min} - C_q}$, normalized by the geometric mean of one or more reference genes [11].

2.3 Statistical analysis

Groups are compared on the linear relative-expression scale—the scale on which values are reported and plotted, and the one used by the R implementation—rather than on ΔC_t; the choice is stated because the two scales give different p-values for the same data, and a single scale is applied throughout the application and the validation. The core library implements Welch and Student two-sample t-tests, the paired t-test, Wilcoxon rank-sum and signed-rank tests, one-way ANOVA with Tukey HSD letters, Cohen's d, IQR-based outlier removal and automatic reference-gene selection by geNorm [17]; the interface exposes the Welch t-test, the rank-sum test and ANOVA with Tukey HSD, while the paired tests are covered by the test suite but not yet offered in the interface. For multi-group designs Tukey's HSD controls the family-wise error rate across the pairwise comparisons of a gene; the reported p-values are per gene and are not additionally corrected across genes, which we state as a limitation rather than leave implicit.

2.4 Software engineering and scope

Results are shown as tables that can be exported as CSV or Excel, with Tukey letters in the table. The application renders the figures in the interface: group means with standard-deviation error bars and significance labels, and the fitted standard curves over the measured dilution series, both exportable as PNG images. Each analysis can also be exported as a plain-text *analysis record* listing the input files and their row counts, the reference gene(s), the control group, the statistical test, the outlier-removal setting, the software version and a timestamp, so that a reader can re-run the analysis without inspecting the session. Quality control is reported as warnings rather than decisions: wells with C_q above a configurable limit (default 35), samples whose technical replicates differ by more than a configurable standard deviation (default 0.5 C_q), and primers with efficiency outside 90–110% are listed on the results page and repeated in the analysis record, while no data are removed automatically. MIQE requires these quantities to be reported rather than to satisfy fixed cut-offs [5], so the defaults are meant to be adapted to the instrument and assay. *qPCRtools* is managed with CMake and ships with unit tests; the interface is bilingual and reads CSV/TXT input, and Excel (`.xlsx`) workbooks when built with the optional OpenXLSX component (the released binaries read CSV/TXT). It is distributed as a single self-contained executable for Windows and macOS—requiring no R, Python or runtime installation—and the source compiles on Linux under the MIT license. Two R functions are not yet ported—the standard-curve slope–intercept quantification (`CalExpCurve`) and the reverse-transcription reagent volume calculation (`CalRTable`)—and are planned for a future release.

3 Results

3.1 Typical analysis workflow

We applied the application to the bundled example dataset (`examples/cq.csv`, `examples/design.csv`): the target gene *fos-glo-myc* against the reference gene *Beta Actin* across three treatment groups (0, 0.5, 1) with three biological replicates each. With the $\Delta\Delta C_t$ method and Welch tests against the control, relative expression rose to 1.92 at dose 0.5 and 2.74 at dose 1, both significantly different from the control ($p = 0.006$ and $p = 0.022$); as defined by Livak and Schmittgen the calibrator is the control group, which is therefore ≈ 1 (here 1.01 ± 0.21). These means, standard deviations and p-values are reproduced without the application or the R package by `validation/check_example_stats.R`, and the complete output is shown in Supplementary Figure S1B. On a four-fold dilution series of four genes, the standard-curve module recovered slopes of -2.07 to -2.24 C_q per dilution step with $R^2 > 0.99$ for three of the four primers (efficiencies 0.85–0.95; Supplementary Figure S1A), illustrating the efficiency estimation that, per MIQE guidance, determines which quantification method is appropriate. The quality-control panel flagged the single $C_q = 40$ well of the weakest primer, and the complete workflow—import, configuration, analysis, export—is performed in the graphical interface without writing code.

3.2 Equivalence to the R package

We ran the C++ implementation and the original R package [7] on identical inputs for every shared method—$\Delta\Delta C_t$ (`CalExp2ddCt`), the standard curve (`CalCurve`) and the efficiency-based RqPCR method (`CalExpRqPCR`)—reading the same on-disk files, so any difference reflects the implementations alone (Supplementary Table S1 and Figure S2). For both expression methods, expression values, standard deviations and standard errors matched to within floating-point precision (maximum relative difference $< 4 \times 10^{-15}$). For the standard curve, slope, intercept, R^2 and efficiency agreed to within 2.5×10^{-3} relative difference, the residual being attributable to the R package rounding its tabulated output to two to four decimals while the underlying C_q inputs are identical; every point falls on the line $y = x$.

The validation is fully reproducible from the open scripts in `validation/` (see Availability). Preparing the port also revealed defects in result aggregation that the R implementation shared—the input of the reference-gene stability measure, replicate handling in the efficiency method and the $\Delta\Delta C_t$ calibrator—as well as one specific to the C++ code (multiple-comparison p-values); each was corrected and is covered by a regression test.

4 Availability and implementation

Implementation: *qPCRtools* is a C++17/Qt6 desktop application built with CMake (≥ 3.20); the core algorithm library requires only Qt Core and optionally links Eigen3, GSL and OpenXLSX, while the interface requires Qt WebEngine and WebChannel. It is distributed as a single self-contained executable for Windows and macOS—no R, Python or other runtime, and no registration, are needed to download or run it—and the source compiles on Linux.

Availability: version 1.2.0 (the version described here; version 1.1.0 is the previous release) is freely available under the MIT license at `https://github.com/lixiang117423/qPCRtools_cpp`, with pre-built binaries in the repository releases. The repository includes documentation, a unit-test suite, example datasets and a fully reproducible equivalence-validation package (`validation/`) comprising the R script, a C++ command-line driver linked against `qpcr_core`, and a comparison script that regenerates all validation outputs and figures. We will keep the software and its source code available for at least two years after publication. Supplementary Figures S1–S2 and Supplementary Table S1 are available online only.

Authors' contributions

XL and HH designed this study and performed the analysis. XL wrote the manuscript. Both authors read and approved the final manuscript.

Acknowledgements

The authors thank the developers of the R packages and libraries on which this work builds, and the users who provided feedback during development.

Funding

This work was supported by the National Key R&D Program of China (2021YFD1000200), the National Natural Science Foundation of China (31801792, 31960554), and the Fok Ying Tung Education Foundation.

Conflict of interest

The authors declare no competing interests.

Use of AI-assisted tools

An AI coding assistant (OpenAI Codex) was used during development for code refactoring, generating unit tests and reviewing the manuscript language. All analyses, numerical results and conclusions were reviewed and verified by the authors, who take responsibility for the content of the article. The same assistant-assisted changes are covered by the regression tests and the equivalence-validation package described above.

References

[1] Mohaned Ahmed and Dae-Ryong Kim. pcr: an R package for quality assessment, analysis and testing of qPCR data. *PeerJ*, 6:e4473, 2018.

[2] Apache Software Foundation. Apache ECharts: A declarative framework for rapid construction of web-based visualization. `https://echarts.apache.org/`, 2024.

[3] Maria Ballester, Raquel Cordón, and Josep M. Folch. DAG Expression: high-throughput gene expression analysis of real-time PCR data using standard curves for relative quantification. *PLoS ONE*, 8(11):e80385, 2013.

[4] Stephen A. Bustin, Jean-François Beaulieu, Jim Huggett, Ramnath Jaggi, Frederick S. B. Kibenge, Pal A. Olsvik, Lois C. Pennington, and Tobias Toegl. MIQE précis: practical implementation of minimum standard guidelines for fluorescence-based quantitative real-time PCR experiments. *BMC Molecular Biology*, 11:74, 2010.

[5] Stephen A. Bustin, Vladimir Benes, Jeremy A. Garson, Jan Hellemans, Jim Huggett, Mikael Kubista, Reinhold Mueller, Tania Nolan, Michael W. Pfaffl, Gregory L. Shipley, et al. The MIQE guidelines: minimum information for publication of quantitative real-time PCR experiments. *Clinical Chemistry*, 55(4):611–622, 2009.

[6] Jan Hellemans, Geert Mortier, Anne De Paepe, Frank Speleman, and Jo Vandesompele. qBase relative quantification framework and software for management and automated analysis of real-time quantitative PCR data. *Genome Biology*, 8(2):R19, 2007.

[7] Xiang Li, Yingmin Wang, Jingyu Li, Xinyue Mei, Yixiang Liu, and Huichuan Huang. qPCRtools: an R package for qPCR data processing and visualization. *Frontiers in Genetics*, 13:1002704, 2022.

[8] Kenneth J. Livak and Thomas D. Schmittgen. Analysis of relative gene expression data using real-time quantitative PCR and the $2^{-\Delta\Delta C_T}$ method. *Methods*, 25(4):402–408, 2001.

[9] Gilles Maussion, Rhalena A. Thomas, Iveta Demirova, Gracia Gu, Eddie Cai, Carol X.-Q. Chen, Narges Abdian, Theodore J. P. Strauss, Sabah Kelaï, Angela Nauleau-Javaudin, Lenore K. Beitel, Nicolas Ramoz, Philip Gorwood, and Thomas M. Durcan. Auto-qPCR; a Python-based web app for automated and reproducible analysis of qPCR data. *Scientific Reports*, 11:21293, 2021.

[10] Stephan Pabinger, Stefan Rödiger, Anja Kriegner, Klemens Vierlinger, and Andreas Weinhäusel. A survey of tools for the analysis of quantitative PCR (qPCR) data. *Biomolecular Detection and Quantification*, 1:23–33, 2014.

[11] Michael W. Pfaffl. A new mathematical model for relative quantification in real-time RT-PCR. *Nucleic Acids Research*, 29(9):e45, 2001.

[12] Michael W. Pfaffl, Graham W. Horgan, and Leo Dempfle. Relative expression software tool (REST) for group-wise comparison and statistical analysis of relative expression results in real-time PCR. *Nucleic Acids Research*, 30(9):e36, 2002.

[13] Christian Ramakers, Jan M. Ruijter, Ronald H. Lekanne Deprez, and Antoon F. M. Moorman. Assumption-free analysis of quantitative real-time polymerase chain reaction (PCR) data. *Neuroscience Letters*, 339(1):62–66, 2003.

[14] Corinne Rancurel, Thuy Van Tran, Corinne Elie, and Frédérique Hilliou. SATQPCR: website for statistical analysis of real-time quantitative PCR data. *Molecular and Cellular Probes*, 46:101418, 2019.

[15] Jan M. Ruijter, Christian Ramakers, Willem M. H. Hoogaars, Yann Karlen, Olaf Bakker, Maurice J. B. van den Hoff, and Antoon F. M. Moorman. Amplification efficiency: linking baseline and bias in the analysis of quantitative PCR data. *Nucleic Acids Research*, 37(6):e45, 2009.

[16] The Qt Company. Qt Cross-Platform Application Development Framework. `https://www.qt.io/`, 2024. Version 6.x.

[17] Jo Vandesompele, Katleen De Preter, Filip Pattyn, Bruce Poppe, Nicole Van Roy, Anne De Paepe, and Frank Speleman. Accurate normalization of real-time quantitative RT-PCR data by geometric averaging of multiple internal control genes. *Genome Biology*, 3(7):research0034, 2002.